

FastDelivery

Plataforma de Delivery Generalista

Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas — PUC Minas, 5º Período

Sprint 1 — Proposta de Domínio e Backend REST | Giuliano Leal B. Pimentel

1. Descrição do Domínio

O FastDelivery é uma plataforma de delivery generalista que conecta clientes a entregadores autônomos. O cliente informa o ponto de origem (onde o item deve ser buscado) e o ponto de destino (onde deve ser entregue), e a plataforma gerencia todo o ciclo de vida da solicitação — da criação até a conclusão ou cancelamento.

O domínio foi escolhido por sua aderência natural aos conceitos explorados na disciplina: comunicação assíncrona entre partes distintas, múltiplos perfis de usuário com fluxos independentes, e necessidade de consistência de estado ao longo das quatro Sprints do semestre.

2. Justificativa

O mercado de entregas sob demanda apresentou crescimento expressivo nos últimos anos, consolidando-se como infraestrutura essencial para o comércio local e individual. Plataformas que conectam diretamente clientes e entregadores autônomos reduzem intermediários, democratizam o acesso ao serviço e geram oportunidades de renda flexível para os prestadores.

Do ponto de vista técnico, o domínio de delivery oferece complexidade crescente e bem delimitada por Sprint: na Sprint 1, um backend REST com controle de estado; na Sprint 2, comunicação assíncrona via mensageria (RabbitMQ ou Redis Pub/Sub); nas Sprints 3 e 4, interfaces móveis em Flutter para cada perfil de usuário. Essa progressão torna o projeto ideal para demonstrar os padrões arquiteturais estudados ao longo de LDAMD.

3. Perfis de Usuário

3.1 Cliente

Usuário que necessita transportar um item de um ponto a outro sem dispor de meios próprios para isso. Interage exclusivamente pelo App Cliente (Flutter — Sprint 3).

- Cria solicitações de entrega informando descrição, origem e destino.
- Acompanha o status da entrega em tempo real.
- Pode cancelar uma solicitação enquanto ainda estiver pendente.

3.2 Prestador (Entregador)

Profissional autônomo que presta o serviço de coleta e entrega. Interage pelo App Entregador (Flutter

— Sprint 4), recebendo notificações de novos pedidos via mensageria na Sprint 2.

- Visualiza a lista de solicitações disponíveis (status pendente).
- Aceita ou recusa uma entrega.
- Atualiza o status conforme a execução: aceito → em_transito → concluído.
- Pode cancelar uma entrega ainda sob sua responsabilidade.

4. Principais Funcionalidades

O backend REST da Sprint 1 expõe os seguintes endpoints:

POST /entregas: Cliente cria uma nova solicitação informando descrição, endereço de origem, endereço de destino e identificador do cliente. Status inicial: pendente.

GET /entregas: Lista todas as solicitações. Suporta filtro opcional por status (?status=pendente | aceito | em_transito | concluído | cancelado).

GET /entregas/<id>: Retorna os detalhes completos de uma entrega. Responde 404 se o id não existir.

PATCH /entregas/<id>/status: Atualiza o status de uma entrega. Valida o valor contra o enum de status aceitos e retorna 400 em caso de valor inválido.

Ciclo de vida de uma entrega: pendente → aceito → em_transito → concluído, com possibilidade de transição para cancelado a partir de qualquer estado anterior à conclusão.

5. Arquitetura Técnica (visão geral)

O backend foi implementado em Python com Flask seguindo Clean Architecture, com separação estrita em quatro camadas:

- Domain: entidade Entrega e enum StatusEntrega com os cinco estados válidos.
- Repositories: acesso ao SQLite sem lógica de negócio (padrão Repository).
- Use Cases: regras de negócio, validações de transição de estado.
- Controllers: rotas Flask finas que delegam inteiramente ao use case.

Nas próximas Sprints serão integrados: mensageria assíncrona via RabbitMQ ou Redis Pub/Sub (Sprint 2), App Cliente em Flutter (Sprint 3) e App Entregador em Flutter (Sprint 4).